

API REST en Rust avec Cargo

Ce TP propose la création d'une API REST *HelloWorld* en langage Rust avec Cargo.

Il permet de mettre en évidence la gestion des crates Rust au travers des fichiers `Cargo.toml`, `Cargo.lock` et des commandes `cargo` associées.

Il démontre également les capacités de gestion d'un projet Rust proposées par `cargo`.

Il propose enfin une brève introduction au langage Rust.

- 1. Prérequis
- 2. Créer un nouveau projet Rust avec Cargo
- 3. Ajouter les dépendances nécessaires
- 4. Créer le code source de l'API
- 5. Lancer et tester l'API
- 6. Gestion de configuration
- 7. Rust / axum avancé : Ajouter des paramètres à la requête
- 8. Annexes
 - 8.1. Documentation utile

1. Prérequis

- Avoir installé Rust (cf. <https://rust-lang.org/tools/install/>).

Installation de Rust sous Linux

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
# Proceed with standard installation...
export PATH="${PATH}:${HOME}/.cargo/bin"
```

- Disposer d'un terminal (Linux/MacOS) ou PowerShell/Command Prompt (Windows)

GitBash pour Windows

Sur Windows, une option rapide et pratique est d'utiliser GitBash.

- Avoir un éditeur de code (VS Code, ...).

Astuce : Extensions Rust pour VS Code

Pour activer le support de Rust dans VS Code, installer l'extension `rust-analyzer` fournie directement par la communauté rust-lang.org.

Penser à relancer VS Code en ligne de commande après avoir étendu la variable d'environnement `PATH`.

2. Créer un nouveau projet Rust avec Cargo

Exécuter les commandes suivantes dans un terminal :

```
cargo new axum_hello_world
cd axum_hello_world
```

Observer les fichiers dans le répertoire :

- Un fichier `Cargo.toml` décrivant le projet Rust.
- Un exemple de fichier source `src/main.rs`.

```
fn main() {
    println!("Hello, world!");
}
```

💡 Avancé : Elements de langage Rust

On rebondit sur l'exemple de code ci-dessus pour discuter quelques concepts spécifiques à Rust :

- Comme en C, le nom de la fonction principale du programme est `main()`.
- Noter la présence du caractère `!` après le symbole `println`.

En Rust, l'exécution des macros se distingue bien des appels de fonction standard grâce à ce caractère `!`, et c'est plutôt une bonne chose.

- Rust est un langage fortement typé, c'est à dire que toute donnée est forcément typée pour le compilateur.

Pour autant, on n'a pas décrit dans le source ce que retourne la fonction `main()`.

Ici, c'est implicite. Comme rien n'est spécifié, cela vaut pour retourner *unit*, i.e. un tuple vide `()`, l'équivalent de `void` en C.

La ligne aurait donc pu s'écrire `fn main() -> () {}`.

Mais ne rien écrire du tout, c'est aussi intuitif et plus lisible.

Jusque là, ça paraît assez simple.

Le fichier source automatiquement créé est un *Hello World* qu'on peut compiler et exécuter dès à présent :

```
cargo run
```

💡 Astuce : `cargo run` et `cargo build`

Rust n'est pas un langage interprété, mais compilé.

Pour autant, la commande `cargo run` suffit car Cargo exécute automatiquement la commande `cargo build` préalable.

Un répertoire `target/` apparaît. À l'intérieur de ce répertoire, un sous-répertoire `target/debug/` contient le binaire compilé.

Observer le répertoire `target/debug/deps/`. Pour l'instant, ce répertoire doit être vide (voire inexistant ? tbc).

Observer le répertoire `~/.cargo/` (répertoire dans lequel Cargo a été installé par `rustup`) :

- À l'intérieur de ce répertoire, on trouve un sous-répertoire `~/.cargo/registry/`.
- Dans le répertoire `~/.cargo/registry/`, on trouve deux sous-répertoires `~/.cargo/registry/cache/` et `~/.cargo/registry/src/`.

Observer le contenu de ces derniers.

Si l'installation de Cargo est récente, ces répertoires devraient être relativement vides à ce stade.

3. Ajouter les dépendances nécessaires

Dans cet exemple, la librairie principale permettant d'implémenter un serveur REST avec Rust est `axum` :

```
cargo add axum
```

Observer les fichiers dans le projet :

- Fichier `Cargo.toml` :
Une section `dependencies` a été ajoutée, indiquant la dépendance `axum` attendue, avec un numéro de version. `typescript` et `@types/node`.
- TODO : Observer le contenu du répertoire `target/debug/deps/`.
- Un fichier `Cargo.lock` est apparu.

Observer son contenu.

On retrouve une structure équivalente à ce qu'on peut trouver dans les fichiers `package-lock.json` avec `npm`.

Observer le contenu des répertoires `~/.cargo/registry/cache/` et `~/.cargo/registry/src/`. Ces répertoires ont été peuplés de données.

📌 Cache local de crates

Le répertoire `~/.cargo/registry/` constitue un cache des crates téléchargées depuis la registry en ligne crates.io.

💡 **Avancé : Elements de langage Rust**

En Rust, on ne parle pas de *packages* mais de *crates*.

Il faut s'y habituer...

Ajouter les autres dépendances nécessaires :

- **tokio** : permet de gérer les traitements asynchrones,
- **serde** constitue une librairie de base pour les SERialisation / DESérialisation d'objets.

```
cargo add tokio serde
```

Lancer une compilation :

```
cargo build
```

puis une exécution :

```
cargo run
```

Toujours le même message de sortie. C'est normal, tant qu'on n'a pas modifié le code.

4. Créer le code source de l'API

Remplacer le contenu du fichier `src/main.rs` par :

```
// src/main.rs

use axum::{routing::get, Router};

#[tokio::main]
async fn main() {
    // Create an axum router.
    let app = Router::new()
        // Handle the '/hello' endpoint.
        .route("/hello", get(|| async {
            // Trace the query.
            println!("/hello requested");
            // Return the result.
            "Hello World!"
        }));
}
```

```

    }));

    // Launch the server on localhost:3000.
    let listener =
tokio::net::TcpListener::bind("localhost:3000").await.unwrap();
    println!("Server started on http://localhost:3000");
    axum::serve(listener, app).await.unwrap();
}

```

💡 Avancé : Elements de langage Rust

L'exemple de code ci-dessus introduit quelques concepts spécifiques à Rust :

- Remarquer la syntaxe `use` avec les accolades et les opérateurs `::`.

Cette syntaxe proposée par le langage permet d'être sélectif dans les symboles importés, en une seule ligne pour une crate de tête donnée.

- Rust est un langage fortement typé, mais on peut se passer de déclarer les types lorsqu'ils peuvent être inférés.

Ici, inutile de dire que la variable `app` est de type `Router` car c'est évident (pour le compilateur du moins) de par la valeur qui lui est affectée.

Cela peut avoir l'avantage d'améliorer la lisibilité du code.

- La gestion des fonctions asynchrones en Rust passe par des mots clés similaires à ceux en JS : `async` et `await`.

A la différence que `await` s'appelle avec un opérateur `.` suite au retour d'un appel asynchrone (et non devant l'appel asynchrone en JS).

- L'utilisation des fonctions asynchrones nécessite le choix et l'instanciation explicite d'un moteur de traitements asynchrones, en l'occurrence `tokio` dans notre exemple.

En effet, Rust étant un langage prévu pour une gestion mémoire fine, la gestion des traitements asynchrone est le genre de considération qui a son importance dans la philosophie du langage.

- Toujours dans l'idée d'une gestion fine de la mémoire, le langage Rust introduit des notions de passages de valeur en *borrow* ou par référence (cf. <https://doc.rust-lang.org/stable/book/ch04-02-references-and-borrowing.html>).

Ces concepts permettent d'assurer la maîtrise d'une notion d'*ownership* sur les mémoires associées aux données (cf. <https://doc.rust-lang.org/stable/book/ch04-01-what-is-ownership.html>).

Pour ces raisons, il est fréquent de voir du code Rust avec une seule instruction (`let app = Router::new();`) sur plusieurs lignes, avec une succession d'appels en chaîne (`.route().route().route()`), design pattern de *builder*.

Un seul appel à `route()` dans cet exemple, mais on pourrait les enchaîner.

- Observer la syntaxe Rust pour la définition d'une fonction *lambda*, qu'on appelle *closure* en Rust.

Les deux caractères `|` correspondent à un début et fin de paramètres, en l'occurrence aucun paramètre dans cet exemple.

- Remarquer que la dernière ligne de la fonction ne se termine pas par un `;`.

En Rust, cela signifie que cette dernière instruction vaut pour une valeur de retour.

C'est donc équivalent à `return "Hello World!";`, mais en plus lisible (selon l'intention du langage, d'accord ou pas d'accord, c'est ainsi).

C'est déjà moins simple. Il faut s'y habituer...

Lancer la compilation :

```
cargo build
```

! Erreur de compilation

La compilation échoue avec une erreur "The default runtime flavor is `multi_thread`, but the `rt-multi-thread` feature is disabled." sur la ligne `#[tokio::main]`.

Ajuster le fichier `Cargo.toml` pour spécifier les *features* activées pour `tokio` et `serde` :

```
serde = { version = "1.0.228", features = ["derive"] }
tokio = { version = "1.49.0", features = ["full"] }
```

Note :

Les versions sont données ci-dessus à titre indicatif.

Conserver les informations de versions présentes au préalable dans le fichier.

Relancer la compilation :

```
cargo build
```

✓ Compilation avec succès

Cargo télécharge des crates supplémentaires puis les compile avec succès.

5. Lancer et tester l'API

Lancer le serveur :

```
cargo run
```

Ouvrir l'adresse "http://localhost:3000/hello" à partir d'un navigateur.

Ou bien en ligne de commande :

```
curl -X GET http://localhost:3000/hello
```

Le résultat obtenu devrait être :

```
Hello World!
```

6. Gestion de configuration

Comme avec **npm** en JS, il existe une option **--lock** pour Cargo qui assure l'utilisation du fichier **Cargo.lock** sans que celui-ci soit modifié.

```
cargo --help
```

cargo --offline et **cargo --frozen**

A noter l'existence de l'option **--offline** qui permet l'utilisation du cache de crates constitué dans le répertoire `~/.cargo/registry/` sans accès à Internet.

A noter également l'existence de l'option **--frozen** qui constitue un raccourci pour la combo des options **--locked** et **--offline**.

On prendra soin de suivre des recommandations équivalentes à ce qu'on a déjà présenté avec **npm** :

Utilisation du fichier Cargo.lock

1. Enregistrer le fichier **Cargo.lock** sous git pour le suivre en version.
2. Utiliser l'option **--locked** pour les commandes **cargo build** et **cargo run** pour assurer le téléchargement et la compilation des dépendances en cohérence avec le fichier **Cargo.lock** enregistré.
3. N'utiliser la commande **cargo update** que pour faire une mise à niveau volontaire des dépendances, et enregistrer le fichier **Cargo.lock** mis à jour en conséquence.

7. Rust / axum avancé : Ajouter des paramètres à la requête

Modifier à nouveau le fichier `src/main.rs` comme suit :

```
// src/main.rs

use axum::{routing::get, Router, extract::Query};
use serde::Deserialize;

// Define '/hello' parameters.
#[derive(Deserialize)]
struct HelloParams {
    // Consider an optional `name` parameter.
    name: Option<String>,
}

/**
 * '/hello' endpoint handler.
 */
async fn hello(Query(params): Query<HelloParams>) -> String {
    // Trace the query.
    println!("/hello requested");

    // Return the result.
    match params.name {
        Some(name) => format!("Hello {}!", name),
        None => "Hello stranger!".to_string(),
    }
}

#[tokio::main]
async fn main() {
    // Create an axum router.
    let app = Router::new()
        // Handle the '/hello' endpoint.
        .route("/hello", get(hello));

    // Launch the server on localhost:3000.
    let listener =
        tokio::net::TcpListener::bind("localhost:3000").await.unwrap();
    println!("Server started on http://localhost:3000");
    axum::serve(listener, app).await.unwrap();
}
```

💡 Avancé : Elements de langage Rust

L'exemple de code ci-dessus introduit encore une fois quelques concepts spécifiques à Rust :

- Plus de *closure* cette fois-ci, on définit et on référence une fonction en bonne et due forme.
- Noter l'application du trait `Deserialize` sur la structure `HelloParams`.

Cela permet d'enrichir la structure `HelloParams` de méthodes de désérialisation, ce qui permet ensuite à `axum` de désérialiser les paramètres de l'URL dans cette structure, pour la passer ensuite à notre handler `hello()`.

- Observer la capacité du langage à déclarer des paramètres ou des variables par *rétro-inférence* (tbc: confirmer le terme de *rétro-inférence* avancé par moi-même d'après ma compréhension).

On écrit `Query(params)` de type `Query<HelloParams>`, mais quand bien même on semble appliquer une fonction ou un constructeur `Query()` sur `params`, c'est bien un paramètre `params` qu'on déclare ici pour la fonction `hello()`, et qu'on utilise ensuite directement dans la fonction.

Magie du langage probablement possible du fait que l'objet `Query` soit un *extracteur* (provient de la crate `axum::extract`).

- Noter le type `Option<String>` pour la donnée `name` de la structure `HelloParams`.

`Option` est considéré comme un *enum* en Rust, i.e. une union de types et/ou de valeurs possibles (définition du concept d'*enum* plus large que dans les autres langages).

En l'occurrence `None` ou `String`.

- Noter en conséquence l'instruction `match` dans la fonction `hello()`, qui en fonction de la valeur de `params.name` retournera la construction de chaîne de caractère correspondant à l'une des deux options `Some(name)` ou `None`.
- Noter encore une fois la déclaration de variable locale `name` par *rétro-inférence* (tbc) de l'écriture de `Some(name)`, qui vaut pour :
 - test que `params.name` n'est pas `None`,
 - et le cas échéant, considère une variable locale `name` de type `String`.
- Noter encore une fois : pas de `;` à la suite du bloc `match`, valant pour valeur de retour de la fonction `hello()`.

Il faut s'y habituer... mais ça vient vite quand on pratique.

Compiler et exécuter :

```
cargo --locked run
```

Tester :

```
curl -X GET http://localhost:3000/hello
curl -X GET http://localhost:3000/hello?name=Alexis%20Royer
```

8. Annexes

8.1. Documentation utile

Documentation officielle Rust :

- Installation : <https://rust-lang.org/tools/install/>
- Doc générale: <https://doc.rust-lang.org/stable/>
- Cargo : <https://doc.rust-lang.org/stable/cargo/index.html>
- Référence du langage Rust : <https://doc.rust-lang.org/stable/book/>

Registry de crates Rust :

- <https://crates.io/>